

GATE 2027 – Computer Science & Information Technology

Complete Syllabus + How to Start

Important: This guide follows the standard GATE CS syllabus structure. Use the official GATE 2027 syllabus PDF as the final authority if any official update differs.

1. General Aptitude

- Verbal Aptitude: grammar, vocabulary, synonyms/antonyms, idioms, sentence completion, reading comprehension, narrative sequencing.
- Quantitative Aptitude: ratios, percentages, powers, logarithms, permutations, combinations, series, basic algebra, geometry, areas/volumes, probability, statistics and data interpretation.

2. Engineering Mathematics

- Discrete Mathematics: propositional and first-order logic, sets, relations, functions, partial orders, graphs, trees, connectivity, matching, graph coloring, combinatorics, recurrence relations and generating functions.
- Linear Algebra: matrices, determinants, systems of linear equations, eigenvalues, eigenvectors and matrix operations.
- Calculus: limits, continuity, differentiability, maxima/minima, mean value theorem and integration.
- Probability & Statistics: probability, conditional probability, Bayes theorem, random variables, expectation, variance, standard deviation and basic discrete distributions.

3. Digital Logic

- Number systems, binary arithmetic, signed representation, 1's/2's complement, floating-point representation.
- Boolean algebra, Boolean functions, SOP/POS, simplification and logic gates.
- Combinational circuits: adders, subtractors, multiplexers, demultiplexers, encoders, decoders and comparators.
- Sequential circuits: latches, flip-flops, counters, registers and memory elements; basic ADC/DAC concepts.

4. Computer Organization & Architecture

- Machine instructions, instruction formats, addressing modes, CPU, ALU, control unit, registers, datapath and control signals.
- Memory hierarchy, RAM/ROM, cache, cache mapping, virtual memory basics and secondary storage.
- Pipelining, pipeline stages, structural/data/control hazards and performance.
- I/O organization, interrupts, DMA and performance measures such as CPU time, CPI, speedup and Amdahl's law.

5. Programming & Data Structures

- C programming: variables, data types, operators, conditions, loops, functions, recursion, arrays, strings, pointers, structures, unions and dynamic memory allocation.
- Data structures: arrays, linked lists, stacks, queues, priority queues, trees, BST, AVL trees, heaps, B-trees and graph representations.
- Tree traversals, BFS and DFS.

6. Algorithms

- Searching and sorting: linear/binary search; bubble, selection, insertion, merge, quick and heap sort; complexity analysis.
- Divide and conquer, recurrences and Master theorem.
- Greedy algorithms: activity selection, fractional knapsack, job sequencing, MST.
- Graph algorithms: BFS, DFS, shortest paths (Dijkstra/Bellman-Ford), Prim, Kruskal and topological sorting.
- Dynamic programming: 0/1 knapsack, LCS, matrix-chain multiplication, optimal substructure and overlapping subproblems.

7. Theory of Computation

- Regular expressions and regular languages; DFA, NFA, epsilon-NFA, conversions and DFA minimization.
- Closure properties and pumping lemma.
- Context-free grammars, derivations, parse trees, ambiguity and normal forms.
- Pushdown automata and CFG/PDA relationship.
- Turing machines, computability, decidability and undecidability including the halting problem.

8. Compiler Design

- Lexical analysis, tokens, regular expressions and finite automata.
- Syntax analysis: recursive descent, LL, LR, SLR and LALR parsing.
- Syntax-directed translation, intermediate code, three-address code, quadruples and triples.
- Runtime environment, activation records, stack/heap allocation.
- Basic blocks, control flow, code generation, register allocation and code optimization.

9. Operating Systems

- Processes, process states, PCB, context switching and threads.
- CPU scheduling: FCFS, SJF, SRTF, Round Robin and priority scheduling; waiting/turnaround/response time.
- Synchronization: race conditions, critical section, mutex, semaphores, monitors and classical synchronization problems.
- Deadlocks: conditions, resource allocation graphs, prevention, avoidance, Banker's algorithm and detection.
- Memory management: paging, segmentation, virtual memory, page tables, TLB and page replacement (FIFO, LRU, Optimal).
- File systems, directories, file allocation, free-space management, inodes and permissions.

10. Databases

- ER model, entities, attributes, relationships, cardinality and participation.
- Relational model, keys and integrity constraints.
- Relational algebra: selection, projection, union, intersection, difference, Cartesian product, joins and division.
- SQL: SELECT, WHERE, GROUP BY, HAVING, ORDER BY, joins, nested queries and aggregate functions.
- Functional dependencies, attribute closure, candidate keys, 1NF, 2NF, 3NF and BCNF.
- Transactions, ACID, serializability, conflict/view serializability, locking, two-phase locking, timestamp protocols and recovery.
- Indexing, B-tree/B+ tree and hashing.

11. Computer Networks

- Network architecture, OSI and TCP/IP models.
- Data link concepts, error detection/correction, flow control, MAC and Ethernet.
- Parity, checksum and CRC.
- IP addressing, IPv4/IPv6, subnetting and CIDR.
- Routing: distance vector and link state.
- ARP, ICMP and basic network protocols.
- TCP/UDP, connection establishment, reliability, flow control and congestion control.
- Application protocols: DNS, HTTP, FTP, SMTP, POP and DHCP.

12. Software Engineering

- Software processes, requirements, design, testing, verification/validation, maintenance, project management, software metrics and Agile basics.

How You Should Start

Do not start all subjects together. Build the foundation first, then move to the high-value core subjects.

Stage	Study order	Main goal
1	Discrete Mathematics → C → Data Structures	Build concepts and problem-solving foundation
2	Algorithms	Complexity + standard algorithms + PYQs
3	OS → DBMS → Computer Networks	Strong core CS preparation
4	COA → Digital Logic	Architecture + numerical practice
5	TOC → Compiler	Automata, grammars and parsing
6	Aptitude + Maths + revision	Keep these running throughout

Your First 2 Weeks

- **Day 1:** Discrete Maths – logic basics + previous-year questions.
- **Day 2:** Logic – propositions, truth tables, logical equivalence.
- **Day 3:** Sets, relations and functions.
- **Day 4:** Counting, permutations and combinations.
- **Day 5:** Graph basics and trees.
- **Day 6:** Revise + solve GATE PYQs from the topics covered.
- **Day 7:** Weekly test + error notebook.
- **Days 8–9:** C basics, arrays, strings and pointers.
- **Days 10–11:** Linked lists, stacks and queues.
- **Day 12:** Trees, BST and heaps.
- **Day 13:** BFS/DFS + basic complexity.
- **Day 14:** Revision + mixed PYQ test.

Daily Routine

- 2 hours: learn/revise concepts
- 1.5 hours: solve GATE PYQs
- 30 minutes: revise formulas/short notes
- 20–30 minutes: General Aptitude
- Keep an error notebook: write the question type, mistake, correct idea and shortcut.

Most Important Rule

Do not wait to finish a subject before solving PYQs. Finish one small topic → immediately solve its GATE PYQs → note mistakes → revise → move on.

For you specifically, start with **Discrete Mathematics**, then move into **C/Data Structures**. You already have college exposure to DSA/DBMS/OS, so later subjects can be revised faster than they would be for a complete beginner.